CI/CD · GitOps · Configuration Management · VDI — Interview Preparation

CI/CD, Automation & VDI Study Guide

One document, every question, full answers with the details and gotchas an interviewer probes for — built for senior SRE, DevOps, and Platform engineers. Covers Jenkins, GitHub Actions, GitLab CI/CD, Argo CD (GitOps), Ansible & Ansible Automation Platform, Azure DevOps, AWS DevOps (Code* / CodeCatalyst), and Virtual Desktop Infrastructure. Filter by tool or difficulty, search any term, and mark questions mastered as you go.

78

questions

8

tools

8

easy

63

medium

7

hard

Mastery

14 of 78 mastered

Search questions, answers, tools (e.g. Jenkinsfile, OIDC, sync wave, FSLogix, buildspec)…

All

78

Jenkins

12

GitHub Actions

10

GitLab CI/CD

8

Argo CD (GitOps)

10

Ansible & AAP

12

Azure DevOps

9

AWS DevOps (Code*)

8

VDI

9

All levels

Easy

Medium

Hard

Expand all

Collapse all

Reset mastery

Showing 78 of 78 questions

01

Jenkins

easy

What is Jenkins, and describe its controller/agent architecture.

Answer

Jenkins is an open-source automation server for CI/CD. Its architecture is controller + agents (formerly master/slave). The controller stores configuration, schedules builds, serves the UI/API, and dispatches work; agents are worker nodes that actually execute build/test/deploy steps. Distributing work to agents lets you scale out and run builds on different OSes/toolchains.

Details & what the interviewer is probing

Best practice: run no builds on the controller itself (set its executors to 0) — builds on the controller are a security and stability risk. Agents connect via SSH, inbound JNLP, or are spun up dynamically (Docker/Kubernetes). The controller is a single point of failure, so protect and back up JENKINS_HOME.

02

Jenkins

medium

Declarative vs scripted pipeline — what's the difference and when do you use each?

Answer

Declarative pipelines use a structured, opinionated syntax (pipeline { agent {} stages {} }) with built-in post, when, environment, and validation — easier to read and the recommended default. Scripted pipelines are full Groovy (node {}) — maximum

flexibility and programmatic control, but more complex and error-prone.

Details & what the interviewer is probing

Use declarative for ~95% of pipelines; drop to scripted (or a script {} block inside declarative) only when you need complex logic. Declarative catches errors earlier and is more maintainable across a team. Both are defined in a Jenkinsfile.

03

Jenkins

medium

What is a Jenkinsfile and pipeline-as-code? Walk through stages, steps, and post.

Answer

A Jenkinsfile is the pipeline definition stored in source control (pipeline-as-code) — versioned, reviewed, and reproducible. It defines stages (logical phases: Build, Test, Deploy) containing steps (individual actions), an agent (where it runs), an environment block (variables/credentials), and post actions (always, success, failure) for notifications/cleanup.

Details & what the interviewer is probing

Storing the pipeline in the repo means CI changes go through PR review alongside code and travel with branches. when gates stages conditionally; parallel runs stages concurrently; post guarantees cleanup/notify regardless of outcome. This is the foundation of treating CI as code.

04

Jenkins

medium

How do agents connect, and how do you scale Jenkins builds?

Answer

Agents attach via SSH (controller connects out), inbound agent/JNLP (agent connects in), or are provisioned dynamically. The modern approach is ephemeral agents: the Docker or Kubernetes plugin spins up a fresh container/pod per build and tears it down after — clean, isolated, and elastic.

Details & what the interviewer is probing

Ephemeral Kubernetes agents are the best-practice for scale: no snowflake build nodes, no leftover state between builds, and you pay only for what you use. Label agents by capability (e.g. linux, docker, gpu) and target stages at the right label.

05

Jenkins

medium

What are Jenkins shared libraries and why use them?

Answer

Shared libraries are reusable pipeline code (Groovy) stored in a separate repo and loaded into many pipelines (@Library). They let you DRY up common logic — standardized build/deploy steps, notifications, security scans — so hundreds of teams call buildAndDeploy() instead of copy-pasting Jenkinsfile blocks.

Details & what the interviewer is probing

Structure: vars/ for global functions (custom steps), src/ for Groovy classes. Versioning the library lets you roll out CI changes centrally. This is how platform teams provide "golden path" pipelines while keeping each team's Jenkinsfile tiny.

06

Jenkins

medium

How do you manage credentials and secrets in Jenkins?

Answer

Use the built-in Credentials plugin to store secrets (passwords, tokens, SSH keys, certs) encrypted in Jenkins, scoped globally or to folders. In pipelines, bind them with withCredentials or the credentials() helper so they're injected as masked environment variables and never printed in logs.

Details & what the interviewer is probing

Better still, integrate an external store (HashiCorp Vault, cloud secret manager) so Jenkins fetches short-lived secrets at run time rather than holding long-lived ones. Scope credentials to folders for least privilege, and never echo secrets or pass them on the command line where they'd appear in process lists/logs.

07

Jenkins

medium

What is a multibranch pipeline, and how should builds be triggered?

Answer

A multibranch pipeline auto-discovers branches/PRs that contain a Jenkinsfile and creates a pipeline for each — so feature branches and PRs get CI automatically. Trigger builds with SCM webhooks (the repo notifies Jenkins on push) rather than SCM polling (Jenkins repeatedly asks the repo) — webhooks are instant and far less load.

Details & what the interviewer is probing

Polling wastes resources and adds latency; webhooks are the right pattern. Multibranch + webhooks means a new branch with a Jenkinsfile is built with zero manual setup, and

stale branch pipelines are cleaned up automatically.

08

Jenkins

hard

What are your Jenkins security best practices?

Answer

Don't build on the controller (0 executors); isolate builds on ephemeral agents.

Enable authentication + authorization (Matrix/Role-Strategy plugin) with least privilege; use folders for team isolation.

Keep script security/sandbox on; approve scripts deliberately.

Store secrets in the Credentials plugin or an external vault; never in Jenkinsfiles.

Keep Jenkins and plugins patched (a major CVE source) and restrict the plugin set.

Use HTTPS, CSRF protection, and audit logging.

Details & what the interviewer is probing

Jenkins's huge plugin surface is its biggest risk — unpatched plugins are a common breach vector. Treat the controller as a high-value target: lock down access, isolate builds, and patch aggressively. Back up JENKINS_HOME for recovery.

09

Jenkins

medium

Jenkins plugins are a strength and a liability — how do you manage them?

Answer

Jenkins's ~1,800-plugin ecosystem lets it integrate with almost anything, but each plugin adds attack surface, upgrade fragility, and potential conflicts. Manage by: installing only what you need, keeping them updated, pinning/testing plugin upgrades in a staging Jenkins, and tracking them as code (e.g. plugins.txt with the Jenkins Docker image / Configuration as Code).

Details & what the interviewer is probing

JCasC (Jenkins Configuration as Code) lets you define the whole controller config (plugins, security, agents) in YAML — reproducible, version-controlled, and DR-friendly. Unmanaged plugin sprawl is a top cause of upgrade breakage and CVEs.

10

Jenkins

medium

How do you back up and recover Jenkins?

Answer

Everything Jenkins needs lives under JENKINS_HOME — job configs, build history, plugins, credentials, and system config. Back it up regularly (snapshot the volume or use a backup plugin) and store it off-box. For recovery, restore JENKINS_HOME onto a fresh controller. Even better, define config as code (JCasC + Jenkinsfiles in repos) so the controller is reproducible from source.

Details & what the interviewer is probing

The "config as code" approach (JCasC + pipelines-in-SCM + plugins.txt in a Docker image) turns Jenkins DR from "restore a fragile snapshot" into "redeploy from Git" — far more reliable. Test restores periodically; an untested backup is a hypothesis.

11

Jenkins

hard

Walk through common Jenkins production problems and how you troubleshoot them.

Answer

Builds stuck in queue — no available executors/agents matching the label, or agents offline. → check agent connectivity and executor counts.

Agent offline / disconnecting — network, JNLP/SSH issues, or agent OOM. → check agent logs and resources.

Controller slow / OOM — too much build history, heavy plugins, builds running on the controller. → tune JVM heap, prune history, offload builds.

Flaky pipelines — non-isolated agents with leftover state. → ephemeral agents.

Details & what the interviewer is probing

Most stability problems trace back to running work on the controller or stateful, shared agents. The fixes — ephemeral containerized agents, zero controller executors, pruned history, JCasC — are the same set that improves security and reproducibility. Use the build's console log + agent log as the first diagnostics.

12

Jenkins

medium

Jenkins vs GitHub Actions / GitLab CI — when would you still choose Jenkins?

Answer

Jenkins is self-hosted, infinitely extensible (plugins/Groovy), and tool/SCM-agnostic — strong for complex, heterogeneous, on-prem, or highly customized pipelines, and where you must control the entire build environment. Cloud-native CI (GitHub Actions, GitLab CI) wins on lower operational overhead, native SCM integration, YAML simplicity, and managed runners.

Details & what the interviewer is probing

The trade-off: Jenkins gives control and flexibility at the cost of you operating it (patching, scaling, plugin management). Many orgs migrate to GitHub Actions/GitLab for new work but keep Jenkins for legacy/complex pipelines. Choose based on operational appetite and integration needs, not hype.

13

GitHub Actions

easy

Explain the core GitHub Actions concepts: workflows, events, jobs, steps, runners.

Answer

Workflow — a YAML file in .github/workflows/ triggered by events (push, pull_request, schedule, workflow_dispatch).

Job — a set of steps that run on one runner; jobs run in parallel by default (use needs for ordering).

Step — an individual action: run a shell command or use a reusable action.

Runner — the machine that executes a job (GitHub-hosted or self-hosted).

Details & what the interviewer is probing

Each job gets a fresh runner environment, so data between jobs must be passed via artifacts or outputs. Actions from the Marketplace are reusable building blocks. The event-driven model means CI/CD is wired directly to repo activity with no separate server to run.

14

GitHub Actions

medium

GitHub-hosted vs self-hosted runners — trade-offs?

Answer

GitHub-hosted runners are managed, ephemeral VMs (clean per job, multiple OSes) — zero maintenance, pay-per-minute. Self-hosted runners are machines you provide — for custom hardware/software, access to private networks, larger/GPU instances, or compliance, but you patch, scale, and secure them.

Details & what the interviewer is probing

Security warning: never use self-hosted runners on public repos — a malicious PR can run arbitrary code on your runner. Make self-hosted runners ephemeral (fresh per job) to avoid state/secret leakage between runs. Use auto-scaling (e.g. Actions Runner Controller on Kubernetes) for elastic self-hosted capacity.

15

GitHub Actions

medium

How do you manage secrets in GitHub Actions and authenticate to the cloud without stored keys?

Answer

Store secrets at repo/org/environment scope; reference them through the secrets context (e.g. secrets.DB_PASSWORD) inside the workflow expression syntax, and they're masked in logs. For cloud access, use OIDC: GitHub issues a short-lived token the cloud trusts (via a configured IdP + role), so you assume an AWS/Azure/GCP role with no long-lived keys stored.

Details & what the interviewer is probing

OIDC is the modern best practice — it eliminates the #1 CI secret-leak risk (static cloud keys). Scope the cloud trust policy to specific repo/branch/environment claims so a fork can't assume the role. Environment secrets + required reviewers gate production credentials behind approvals.

16

GitHub Actions

medium

What is a matrix build and when is it useful?

Answer

A matrix strategy runs the same job across combinations of parameters — e.g. test on Node 18/20/22 × Ubuntu/Windows/macOS — fanning out into parallel jobs from one definition. It's ideal for testing across versions, OSes, or configurations without duplicating job code.

Details & what the interviewer is probing

Use fail-fast to stop the whole matrix on first failure (or disable it to see all results), include/exclude to tweak specific combinations, and max-parallel to cap concurrency. Matrix builds are the clean way to get broad compatibility coverage cheaply.

17

GitHub Actions

medium

Reusable workflows vs composite actions — what's the difference?

Answer

Both promote DRY reuse. A composite action bundles multiple steps into one reusable step (good for a sequence like setup-and-cache). A reusable workflow (workflow_call) is an entire callable workflow with its own jobs, inputs, secrets, and outputs — good for standardizing whole build/deploy pipelines across repos.

Details & what the interviewer is probing

Composite action = reuse at the step level inside a job; reusable workflow = reuse at the workflow/job level across repos. Platform teams publish reusable workflows so every service repo calls one standardized pipeline, similar to Jenkins shared libraries.

18

GitHub Actions

medium

How do caching and artifacts work, and how do they differ?

Answer

Caching (actions/cache) persists dependencies (npm/pip/Maven caches, Docker layers) between runs keyed by a hash of the lock file — to speed up builds. Artifacts (upload/download-artifact) pass build outputs between jobs in the same run, or retain them for download (test reports, binaries).

Details & what the interviewer is probing

Mental model: cache = speed up future runs (transient, reconstructible); artifacts = share/keep outputs of this run. Cache keys must include the dependency lock hash so a dependency change busts the cache. Don't cache secrets or large reconstructible build outputs.

19

GitHub Actions

medium

What are environments and protection rules in GitHub Actions?

Answer

Environments (e.g. staging, production) group deployment config and add protection rules: required reviewers (manual approval before a job targeting that environment runs), wait timers, and branch restrictions. Environment-scoped secrets are only available to jobs deploying to that environment.

Details & what the interviewer is probing

This is how you gate production: a deploy job referencing environment: production pauses for approval and only then gets the prod credentials. It brings change control and least-privilege secret access into the pipeline without external tooling.

20

GitHub Actions

medium

What is GITHUB_TOKEN and how do you apply least privilege to it?

Answer

GITHUB_TOKEN is an automatically-generated, short-lived token scoped to the repo that workflows use to call the GitHub API (e.g. post comments, create releases). By default it can be broad, so set permissions: at the workflow/job level to grant only what's needed (e.g. contents: read), following least privilege.

Details & what the interviewer is probing

Default-restrict the token (org setting "read-only by default") and elevate per job only as required — a compromised step with a write-all token is dangerous. The token is automatically revoked when the job ends, which limits exposure compared to a stored PAT.

21

GitHub Actions

hard

What are the supply-chain risks with third-party Actions, and how do you mitigate them?

Answer

A third-party action runs code in your pipeline with access to your secrets and token — a compromised or malicious action (or a hijacked tag) can exfiltrate them. Mitigate by pinning actions to a full commit SHA (not a mutable tag like @v3), vetting/auditing actions, using verified/first-party actions where possible, restricting GITHUB_TOKEN permissions, and limiting which actions are allowed org-wide.

Details & what the interviewer is probing

Pinning to a SHA is the single most important control — a tag can be moved to malicious code, a SHA cannot. Use Dependabot to update pinned SHAs deliberately. Treat actions like any dependency in your supply chain: pin, scan, and minimize.

22

GitHub Actions

medium

How do you control job ordering, conditionals, and concurrency?

Answer

Ordering: jobs run in parallel unless you declare needs: (job B waits for A) — this also forms a DAG and passes outputs. Conditionals: if: expressions gate jobs/steps (e.g. only deploy on the main branch or on a tag). Concurrency: a concurrency group cancels or queues superseded runs (e.g. cancel in-progress deploys for the same branch).

Details & what the interviewer is probing

concurrency prevents two deploys of the same environment from racing — a common production safety control. needs + job outputs build multi-stage pipelines; if with branch/tag/event context implements promotion logic (build everywhere, deploy only

from main).

23

GitLab CI/CD

easy

Describe the structure of a .gitlab-ci.yml pipeline.

Answer

The .gitlab-ci.yml at the repo root defines the pipeline. Jobs are assigned to stages (e.g. build, test, deploy) that run sequentially; jobs within a stage run in parallel. Each job specifies a script, optionally an image (Docker), rules/only/except for when it runs, and artifacts/cache.

Details & what the interviewer is probing

If any job in a stage fails, later stages don't run (unless allow_failure). rules is the modern, flexible replacement for only/except. GitLab CI is built into the GitLab platform, so pipelines live next to code, issues, and merge requests with no separate CI server.

24

GitLab CI/CD

medium

What are GitLab Runners and executors?

Answer

Runners are the agents that execute jobs. They can be shared (across the instance), group, or project-specific. Each runner uses an executor that defines how jobs run: docker (each job in a fresh container — most common), shell (directly on the runner host), kubernetes (a pod per job), and others.

Details & what the interviewer is probing

The Docker and Kubernetes executors give clean, isolated, ephemeral builds — preferred over the shell executor (which leaves state and runs as the runner user). The Kubernetes executor autoscales elastically. Register your own runners for private networks, special hardware, or compliance.

25

GitLab CI/CD

medium

What pipeline types does GitLab support (basic, DAG, parent-child, MR pipelines)?

Answer

Basic/stage-based — jobs grouped into sequential stages.

DAG (needs:) — jobs start as soon as their dependencies finish, ignoring stage boundaries, for faster pipelines.

Parent-child — a pipeline triggers separate child pipelines (good for monorepos: run only the affected component's pipeline).

Merge request pipelines — run against the MR, with features like merged-results and merge trains.

Details & what the interviewer is probing

DAG (needs) is the key performance lever — a slow test stage shouldn't block an unrelated deploy-prep job. Merge trains serialize merges with combined testing to prevent "green-on-branch, broken-on-main." Parent-child pipelines keep monorepo CI fast and scoped.

26

GitLab CI/CD

medium

How do environments, deployments, and review apps work in GitLab?

Answer

A job can declare an environment (e.g. staging, production), and GitLab tracks deployments to it (history, current version, rollback). Review apps spin up a temporary, per-merge-request environment so reviewers can see the change live, then tear it down when the MR closes.

Details & what the interviewer is probing

Environments give deployment visibility and one-click rollback to a previous deployment. Review apps are powerful for frontend/UX review — every MR gets a disposable live preview. Protected environments add required approvals before deploys.

27

GitLab CI/CD

medium

How do CI/CD variables and secrets work in GitLab?

Answer

Define variables at the instance/group/project level or inline. Mark them masked (hidden in job logs) and protected (only exposed to jobs on protected branches/tags) for secrets. Reference them as $VAR_NAME in scripts. For stronger secret management, integrate HashiCorp Vault or use OIDC to fetch cloud credentials at run time.

Details & what the interviewer is probing

"Protected" is crucial — it stops a feature branch (or a fork MR) from accessing production secrets. Masking prevents accidental log leakage but isn't real encryption, so combine protected + masked + Vault/OIDC for production credentials.

28

GitLab CI/CD

medium

How do you keep GitLab CI configuration DRY (include, templates, components)?

Answer

Use include: to pull in CI config from other files/repos/templates, extends: and YAML anchors to share job definitions, and CI/CD components (the newer, versioned, reusable pipeline building blocks from the catalog) to standardize pipelines across many projects.

Details & what the interviewer is probing

This is GitLab's answer to "don't copy-paste pipelines everywhere," analogous to reusable workflows (GHA) or shared libraries (Jenkins). A platform team publishes components/templates; project pipelines include them and stay tiny and consistent.

29

GitLab CI/CD

medium

What built-in DevSecOps scanning does GitLab provide?

Answer

GitLab bundles security scanning into the pipeline: SAST (static code analysis), DAST (dynamic/running-app testing), dependency scanning (vulnerable libraries), container scanning, secret detection, and license compliance. Findings surface in the merge request and a security dashboard, gating risky changes before merge.

Details & what the interviewer is probing

This integrated "shift-left" security is a major GitLab differentiator — you enable templates rather than wiring separate tools. Results appear inline on the MR so developers fix issues in context. Vulnerability management and compliance dashboards aggregate findings across projects.

30

GitLab CI/CD

medium

GitLab CI vs GitHub Actions — how do they compare?

Answer

Both are YAML-based, repo-integrated CI/CD. GitLab is a single integrated DevSecOps platform (SCM + CI + built-in security + container registry + environments), strong for an all-in-one self-managed or SaaS suite. GitHub Actions has a massive Marketplace of reusable actions and the largest community, with CI/CD layered onto GitHub's dominant SCM.

Details & what the interviewer is probing

GitLab leans "batteries-included platform" (especially self-managed and built-in security); GitHub leans "ecosystem + marketplace." Choice usually follows where your code already lives and whether you want an integrated suite vs a composable, action-rich ecosystem.

31

Argo CD (GitOps)

easy

What is Argo CD and what is GitOps?

Answer

Argo CD is a declarative, GitOps continuous-delivery tool for Kubernetes. GitOps means Git is the single source of truth for desired cluster state; an in-cluster agent (Argo CD) continuously compares the live cluster to the manifests in Git and reconciles them. You change infrastructure by merging a PR, not by running kubectl apply manually.

Details & what the interviewer is probing

This is a pull-based model: Argo CD runs inside the cluster and pulls from Git, so no external CI system needs cluster credentials. Benefits: full audit trail (Git history), easy rollback (git revert), drift detection, and declarative reproducibility.

32

Argo CD (GitOps)

medium

Describe the Argo CD Application resource.

Answer

An Application CRD ties together: a source (a Git repo, a path, and a revision — plain manifests, Helm, or Kustomize) and a destination (a target cluster and namespace). Argo CD reconciles that source into that destination, tracking sync and health.

Details & what the interviewer is probing

Multiple sources/tools are supported (Helm values, Kustomize overlays per environment). Applications are grouped into Projects for RBAC and guardrails (which repos/clusters/namespaces an app may use). The Application is the unit Argo CD manages and reports on.

33

Argo CD (GitOps)

medium

Sync status vs health status — what's the difference?

Answer

Sync status answers "does the live cluster match Git?" — Synced or OutOfSync (someone changed Git or the cluster, so they diverge). Health status answers "are the deployed resources actually healthy?" — Healthy, Progressing, Degraded, etc. (e.g. a Deployment whose pods aren't ready).

Details & what the interviewer is probing

They're orthogonal: an app can be Synced but Degraded (Git applied, but the pods are crashing) or OutOfSync but Healthy (running fine, but Git has a newer version not yet applied). You watch both — sync for drift, health for runtime correctness.

34

Argo CD (GitOps)

medium

Manual vs automated sync — and what do prune and self-heal do?

Answer

Manual sync requires a human/pipeline to trigger applying Git changes. Automated sync applies them automatically when Git changes. Two options refine it: prune deletes resources removed from Git (otherwise orphaned resources linger), and self-heal reverts any manual/live changes back to Git state (enforcing Git as truth).

Details & what the interviewer is probing

Automated + prune + self-heal gives true GitOps enforcement — the cluster can't drift from Git for long. Be deliberate with prune (a bad delete in Git removes resources) and self-heal (it will undo emergency manual hotfixes). Many teams auto-sync non-prod and gate prod with manual sync + approvals.

35

Argo CD (GitOps)

medium

What is the App of Apps pattern?

Answer

App of Apps is a parent Argo CD Application whose Git source contains the manifests for other Applications. Syncing the parent creates/manages all the child apps — letting you bootstrap and manage an entire cluster's set of applications from one root, declaratively.

Details & what the interviewer is probing

It's how teams manage many apps/add-ons (ingress, monitoring, cert-manager, workloads) as a unit and bootstrap new clusters reproducibly. ApplicationSets are a more powerful, templated successor for generating many apps across clusters/environments from generators.

36

Argo CD (GitOps)

medium

What are sync waves and resource hooks?

Answer

Sync waves order resource application within a sync (lower wave numbers apply first) — e.g. create a namespace and CRDs before the workloads that depend on them. Resource hooks run jobs at sync phases: PreSync (e.g. a DB migration before deploy), Sync, and PostSync (e.g. smoke tests/notifications).

Details & what the interviewer is probing

Waves + hooks let GitOps handle ordering and lifecycle tasks that plain "apply everything at once" can't — database migrations, dependency ordering, and post-deploy verification. A failed PreSync hook can halt the sync, protecting you from deploying on a bad migration.

37

Argo CD (GitOps)

medium

What are ApplicationSets and why use them?

Answer

ApplicationSet is a controller that generates many Argo CD Applications from a template plus a generator — e.g. one app per cluster (cluster generator), per Git directory (monorepo of apps), or per list element. It removes the toil of hand-writing dozens of near-identical Application manifests.

Details & what the interviewer is probing

The classic use: deploy the same app to many clusters/regions/environments from a single templated definition, or onboard every microservice folder in a monorepo automatically. It's the scalable evolution of the App-of-Apps pattern for fleet/multi-tenant GitOps.

38

Argo CD (GitOps)

medium

How does Argo CD detect and handle drift?

Answer

Argo CD continuously compares live cluster state to Git. Any divergence (a manual kubectl edit, an operator changing something) shows the app as OutOfSync and highlights the exact diff. With self-heal enabled, Argo CD automatically reverts the live change back to the Git-declared state.

Details & what the interviewer is probing

This makes unauthorized/manual changes visible and (optionally) auto-corrected — a big reliability and security win. The trade-off: legitimate emergency manual fixes get reverted unless you also update Git, which reinforces the discipline of "change Git, not the cluster."

39

Argo CD (GitOps)

medium

Argo CD vs Flux — and why is the pull model preferred?

Answer

Both are CNCF GitOps tools that reconcile Git to Kubernetes. Argo CD has a rich UI, a clear Application model, projects/RBAC, and ApplicationSets. Flux is more controller/Kubernetes-native and composable (toolkit of controllers), often favored for a CLI/GitOps-toolkit approach. Both run in-cluster and pull from Git.

Details & what the interviewer is probing

The pull model means no external CI system holds cluster credentials (the agent pulls), you get built-in drift detection and self-heal, and rollback is a Git revert. Contrast with push CD, where a pipeline needs kube credentials and there's no continuous reconciliation. Choice between Argo/Flux is largely UX/ecosystem preference.

40

Argo CD (GitOps)

hard

How does Argo Rollouts complement Argo CD, and how do you handle secrets in GitOps?

Answer

Argo CD syncs declared state but does plain rolling updates. Argo Rollouts is a separate controller providing progressive delivery — canary and blue-green deployments with automated analysis (metric-based promotion/abort) and traffic shaping via a service mesh/ingress. You use Argo CD to deploy a Rollout resource, and Rollouts manages the safe ramp.

Secrets in GitOps: never commit plaintext secrets to Git — use Sealed Secrets, SOPS, or the External Secrets Operator (pulls from Vault/cloud secret managers at runtime).

Details & what the interviewer is probing

Rollouts adds the risk control (canary + automated rollback on bad metrics) that vanilla GitOps lacks. The GitOps secrets problem is real: encrypt-before-commit (Sealed Secrets/SOPS) or reference-don't-store (External Secrets). Putting raw secrets in a Git repo — even private — is the classic GitOps mistake.

41

Ansible & AAP

easy

What is Ansible, and what do 'agentless', 'push-based', and 'idempotent' mean?

Answer

Ansible is an automation tool for configuration management, app deployment, and orchestration. Agentless — it needs no software installed on managed nodes; it connects over SSH (or WinRM for Windows). Push-based — the control node pushes changes out to targets (vs a pull agent). Idempotent — running a playbook repeatedly converges to the same state without making changes if already correct.

Details & what the interviewer is probing

Agentless is a big operational advantage — nothing to install/patch on hundreds of nodes (just SSH + Python). Idempotency comes from Ansible's modules describing desired state, not commands. This makes Ansible safe to run repeatedly, the foundation of reliable automation.

42

Ansible & AAP

medium

Describe playbook structure: plays, tasks, modules, and handlers.

Answer

A playbook (YAML) contains one or more plays, each targeting a group of hosts and running an ordered list of tasks. Each task calls a module (e.g. apt, copy, service) with parameters. Handlers are special tasks triggered by notify only when a task reports a change (e.g. restart a service after its config file changes).

Details & what the interviewer is probing

Modules are the idempotent building blocks (they do the right thing or nothing). Handlers run once, at the end, even if notified multiple times — perfect for "restart service only if config actually changed." Plays map hosts → roles/tasks; tasks map to modules.

43

Ansible & AAP

medium

How does Ansible inventory work (static vs dynamic)?

Answer

The inventory defines the hosts Ansible manages, organized into groups (e.g. [web], [db]) with per-host/group variables. Static inventory is a file (INI/YAML) you maintain. Dynamic inventory uses plugins/scripts that query a source (AWS, Azure, vSphere) at run time, so the host list reflects live cloud reality automatically.

Details & what the interviewer is probing

Dynamic inventory is essential in the cloud — instances come and go, so you discover them by tags/attributes rather than hand-maintaining IPs. Group variables (group_vars/, host_vars/) let you parameterize behavior per environment/role cleanly.

44

Ansible & AAP

medium

What are roles and Ansible Galaxy?

Answer

A role is a standardized, reusable bundle of automation with a defined directory layout — tasks/, handlers/, templates/, files/, vars/, defaults/, meta/. Roles package a capability (e.g. "install and configure nginx") for reuse across playbooks. Ansible Galaxy is the public hub for sharing and downloading community roles and collections.

Details & what the interviewer is probing

Roles are how you keep large automation DRY and maintainable — playbooks become thin lists of roles applied to host groups. defaults/ holds overridable defaults, vars/ holds higher-precedence role vars. Pin roles/collections from Galaxy with requirements.yml for reproducibility.

45

Ansible & AAP

medium

How does Ansible achieve idempotency, and why are command/shell modules a caveat?

Answer

Most modules are idempotent — they check current state and only change what's needed (service ensures running; if already running, no change). But the command/shell modules run raw commands and are not idempotent by default — they execute every run. Use a proper module when one exists; if you must use shell, guard it with creates/removes or changed_when/when conditions.

Details & what the interviewer is probing

The classic anti-pattern is doing everything with shell, losing idempotency and reporting accuracy. Prefer purpose-built modules; they make playbooks safe to re-run and report changes truthfully. changed_when: false marks read-only commands so they don't show spurious "changed."

46

Ansible & AAP

medium

Explain variables, facts, and Jinja2 templating in Ansible.

Answer

Variables parameterize playbooks (defined in inventory, group_vars/host_vars, playbooks, roles, or via --extra-vars, following a precedence order). Facts are system data Ansible auto-gathers about each host (OS, IP, memory) usable as variables. Jinja2 templates ({{ variable }}) render config files dynamically per host via the template module.

Details & what the interviewer is probing

Variable precedence is a common interview topic — roughly: role defaults (lowest) → inventory/group/host vars → play vars → --extra-vars (highest). Facts let one playbook adapt per host (e.g. choose apt vs yum by OS family). Templating turns static config into per-environment-rendered config.

47

Ansible & AAP

medium

How do you manage secrets with Ansible Vault?

Answer

Ansible Vault encrypts sensitive data (variables or whole files) at rest so you can safely commit them to source control. You encrypt with ansible-vault encrypt and supply the password (or a vault-id) at run time with --ask-vault-pass or a password file/script. Encrypted vars decrypt in memory during the run.

Details & what the interviewer is probing

For stronger secret management, integrate an external store (HashiCorp Vault, cloud secret managers) and fetch secrets at run time instead of storing encrypted blobs in Git. Keep the vault password itself out of the repo (CI secret/credential store). Vault solves "secrets in version control," not full secret lifecycle.

48

Ansible & AAP

medium

What do handlers, loops, conditionals (when), and tags give you?

Answer

Handlers run on change notification (restart-on-change). Loops (loop:) repeat a task over a list (install many packages, create many users). Conditionals (when:) run a task only if a condition holds (e.g. only on RedHat hosts). Tags let you run a subset of a playbook (--tags deploy) without executing everything.

Details & what the interviewer is probing

These make playbooks data-driven and flexible: loop over inventory/vars, branch on facts, and selectively run parts of a large playbook. Tags are great for big playbooks where you want to re-run just the "config" or "deploy" portion quickly.

49

Ansible & AAP

medium

What does Ansible Automation Platform (Tower/AWX) add over plain Ansible?

Answer

Plain Ansible is a CLI. Ansible Automation Platform (formerly Tower; AWX is the upstream open-source project) adds an enterprise control layer: a web UI and REST API, RBAC, job templates with surveys (parameterized self-service), scheduling, centralized credential management, inventory management, workflows (chaining jobs), logging/auditing, and notifications.

Details & what the interviewer is probing

It turns Ansible from "engineers run playbooks from their laptops" into a governed, auditable, self-service automation service — non-experts can launch curated job templates with guardrails, credentials are stored centrally (not on laptops), and everything is logged. This is what makes Ansible enterprise-grade at scale.

50

Ansible & AAP

medium

Ansible vs Terraform — how do they differ and when do you use each?

Answer

Terraform is declarative provisioning (create cloud infrastructure) with explicit state and an immutable-infrastructure bent. Ansible is primarily configuration management (configure existing servers, deploy apps) — procedural/ordered, agentless, and mutable (it changes machines in place). Terraform builds the servers; Ansible configures what's on them.

Details & what the interviewer is probing

They're complementary: Terraform provisions the VMs/network, Ansible configures the OS/apps (or you bake images with Packer for immutable infra). Ansible can provision

cloud resources, but Terraform's state/plan model is stronger for that; Terraform can't do in-place config management well. Common pattern: Terraform + Ansible (or Terraform + immutable images).

51

Ansible & AAP

medium

How do you test/dry-run Ansible, and what are ad-hoc commands?

Answer

Check mode (--check) does a dry run, reporting what would change without changing anything (pair with --diff to see content diffs). Ad-hoc commands run a single module against hosts without a playbook — ansible web -m ping or ansible web -m service -a "name=nginx state=restarted" — for quick one-off tasks.

Details & what the interviewer is probing

Check mode is your safety net before applying to production (note: not all modules fully support it). Ad-hoc commands are great for quick checks/fixes (ping all hosts, gather a fact, restart a service) without authoring a playbook. For real testing, use Molecule to test roles in containers.

52

Ansible & AAP

hard

How do you scale Ansible performance and troubleshoot slow/failing runs?

Answer

Performance levers: increase forks (parallelism across hosts), use strategies (free lets hosts run independently instead of lock-step linear), enable fact caching and gather_facts: false when facts aren't needed, use pipelining to reduce SSH operations, and limit scope with --limit. Troubleshoot with -vvv (verbose), --check --diff, and --start-at-task.

Details & what the interviewer is probing

Fact gathering and SSH round-trips dominate runtime at scale — caching facts and pipelining help a lot. Idempotency issues usually come from shell/command tasks; fix with proper modules or changed_when. For very large fleets, AAP/AWX with smart inventory and a control-node fleet scales better than a single laptop.

53

Azure DevOps

easy

What are the core Azure DevOps services?

Answer

Boards — work tracking (backlogs, sprints, Kanban, work items).

Repos — Git (and legacy TFVC) source control.

Pipelines — CI/CD (build and release).

Test Plans — manual/exploratory test management.

Artifacts — package feeds (NuGet, npm, Maven, Python, universal).

Details & what the interviewer is probing

It's an integrated end-to-end DevOps suite (plan → code → build → test → release → package). Many teams use a subset (commonly Repos + Pipelines, or Boards + Pipelines while hosting code on GitHub). Pipelines can build from Azure Repos or GitHub and other sources.

54

Azure DevOps

medium

YAML vs Classic pipelines, and the stages/jobs/steps model?

Answer

YAML pipelines are pipeline-as-code (a azure-pipelines.yml in the repo) — versioned, reviewable, and the recommended approach. Classic pipelines are configured in the UI (no code) — easier to start, harder to version. The model: a pipeline has stages → jobs (each runs on an agent) → steps (tasks/scripts).

Details & what the interviewer is probing

Prefer YAML for the same reasons as any pipeline-as-code: review, history, branch-awareness, reuse via templates. Classic release pipelines are still common for legacy gated releases. Jobs in a stage can run in parallel; stages run sequentially with optional approvals between them.

55

Azure DevOps

medium

Microsoft-hosted vs self-hosted agents?

Answer

Pipelines run on agents in agent pools. Microsoft-hosted agents are managed, clean-per-run VMs (Windows/Linux/macOS) with common tooling — no maintenance, but limited customization and a concurrency/time cap. Self-hosted agents run on your machines — for custom software, private-network access, special hardware, or longer/faster builds — but you maintain and secure them.

Details & what the interviewer is probing

Use Microsoft-hosted for standard builds; self-hosted when you need private resources, specific tools/hardware, or higher throughput. Make self-hosted agents ephemeral where possible. Scale-set agents (backed by an Azure VM scale set) give elastic self-hosted capacity.

56

Azure DevOps

medium

What are service connections, and how should they authenticate to Azure?

Answer

A service connection lets a pipeline access an external system (Azure, AWS, a registry, Kubernetes). For Azure, it historically used a service principal with a client secret; the modern, recommended option is workload identity federation (OIDC) — the pipeline gets short-lived tokens, so there's no stored secret to rotate or leak.

Details & what the interviewer is probing

Workload identity federation is the Azure DevOps equivalent of GitHub Actions OIDC — adopt it to eliminate long-lived service-principal secrets. Scope the service connection to the least privilege (specific resource group/subscription) and restrict which pipelines may use it.

57

Azure DevOps

medium

How do variable groups, secret variables, and Key Vault integration work?

Answer

Variables parameterize pipelines; mark them secret to encrypt and mask them in logs. Variable groups (in the Library) share variables across pipelines, and a variable group can be linked to Azure Key Vault so secrets are pulled from Key Vault at run time rather than stored in the pipeline.

Details & what the interviewer is probing

Key Vault linkage is the strong pattern — secrets live in Key Vault (rotated, access-controlled, audited) and the pipeline fetches them via the service connection's identity. Secret variables aren't passed to scripts as plain env vars unless explicitly mapped, which prevents accidental leakage.

58

Azure DevOps

medium

How do environments, approvals/checks, and deployment strategies work?

Answer

Environments (e.g. dev, prod) are deployment targets with approvals and checks — required approvers, business-hours gates, or invoking a function/REST check before a deploy proceeds. Deployment jobs support strategies: runOnce, rolling, and canary, with lifecycle hooks (preDeploy, deploy, routeTraffic, postRouteTraffic, on:failure).

Details & what the interviewer is probing

Approvals + checks bring change control into the pipeline — production deploys pause for sign-off and can require security/quality gates. The canary/rolling strategies (with hooks) implement progressive delivery natively. Environments also give deployment history and traceability per target.

59

Azure DevOps

medium

How do you reuse pipeline logic with templates?

Answer

Azure Pipelines templates let you factor out reusable YAML — step, job, stage, and variable templates — referenced from multiple pipelines (even across repos). Parameters make them configurable. This standardizes build/deploy logic so each pipeline stays small and consistent.

Details & what the interviewer is probing

Templates are Azure DevOps's DRY mechanism (analogous to GHA reusable workflows / GitLab includes / Jenkins shared libraries). Required templates can also enforce security/compliance steps that pipelines must include — a governance control for platform teams.

60

Azure DevOps

medium

What does Azure Artifacts provide?

Answer

Azure Artifacts hosts package feeds for NuGet, npm, Maven, Python (PyPI), and universal packages — a private registry for your internal packages and a cache/proxy (upstream sources) for public ones. Pipelines publish build outputs to a feed and consume dependencies from it.

Details & what the interviewer is probing

Upstream sources let you proxy public registries through your feed (caching + a single trusted source + protection against public outages/typosquatting). Views (e.g. @prerelease, @release) promote package quality levels. It's the AzDO analogue of

CodeArtifact / a private Nexus/Artifactory.

61

Azure DevOps

medium

Azure DevOps vs GitHub Actions — both are Microsoft; when which?

Answer

Azure DevOps is a mature, integrated suite (Boards, Repos, Pipelines, Test Plans, Artifacts) with strong enterprise governance, classic release pipelines, and Test Plans — favored by enterprises already invested in it. GitHub Actions is lighter, community/marketplace-driven, tightly coupled to GitHub's dominant SCM, and where Microsoft is investing for new work.

Details & what the interviewer is probing

Microsoft positions GitHub as the strategic platform, so greenfield projects often start there; many enterprises keep Azure DevOps for its end-to-end suite, Boards, Test Plans, and existing pipelines. They interoperate (Azure Pipelines can build GitHub repos). Choice follows existing investment and whether you need the full suite vs the GitHub ecosystem.

62

AWS DevOps (Code*)

easy

Give an overview of the AWS developer tools (Code* suite) and CodeCatalyst.

Answer

CodeCommit — managed Git repos (note: AWS restricted it to existing customers in 2024; new projects usually use GitHub/GitLab).

CodeBuild — managed build/test service.

CodeDeploy — automated deployments to EC2, ECS, and Lambda.

CodePipeline — orchestrates the end-to-end CI/CD workflow.

CodeArtifact — package repository.

CodeCatalyst — a newer unified DevOps service (projects, repos, workflows, environments) layered on these.

Details & what the interviewer is probing

The Code* tools are managed, IAM-integrated, and pay-per-use — attractive when you want CI/CD inside AWS with no servers to run. Many teams mix-and-match: GitHub for SCM + CodePipeline/CodeBuild/CodeDeploy for build/deploy. Confirm CodeCommit availability before relying on it for new projects.

63

AWS DevOps (Code*)

medium

What is CodeBuild and what's in a buildspec?

Answer

CodeBuild is a fully managed build service — it compiles source, runs tests, and produces artifacts in ephemeral, containerized build environments, scaling automatically with no build servers to manage. The build is defined by a buildspec.yml with phases (install, pre_build, build, post_build) plus artifacts and cache sections.

Details & what the interviewer is probing

You pay per build-minute, and builds run in isolated containers (clean per run). The buildspec lives in the repo (build-as-code). Use the cache section + S3/local caching to speed up dependency-heavy builds, and grant the build's IAM role least privilege for what it needs (e.g. push to ECR).

64

AWS DevOps (Code*)

hard

What is CodeDeploy, and how do in-place vs blue-green deployments work?

Answer

CodeDeploy automates deployments to EC2/on-prem, ECS, and Lambda, driven by an appspec file (hooks like BeforeInstall, AfterInstall, ApplicationStart, ValidateService). In-place updates existing instances (taking some out of service during update — risk of downtime, slower rollback). Blue-green provisions a new (green) fleet, shifts traffic once healthy, and keeps blue for instant rollback.

Details & what the interviewer is probing

For Lambda/ECS, CodeDeploy does canary/linear traffic shifting (e.g. 10% then 90%) with CloudWatch alarm-based automatic rollback. Blue-green minimizes risk and rollback time at the cost of extra capacity. Deployment configs (e.g. OneAtATime, HalfAtATime, canary percentages) control the rollout pace.

65

AWS DevOps (Code*)

medium

What does CodePipeline orchestrate, and what are stages and actions?

Answer

CodePipeline models a release as stages (Source → Build → Test → Deploy) containing actions. It reacts to a source change (CodeCommit/GitHub/S3), runs a build (CodeBuild), optional manual-approval actions, and deploys (CodeDeploy/CloudFormation/ECS). It's the glue connecting the other Code* services

and third-party tools.

Details & what the interviewer is probing

It's event-driven and managed (no orchestrator to run). Manual-approval actions add gates; parallel actions speed stages; it integrates with CloudFormation for IaC deploys and with Lambda for custom actions. Each action assumes an IAM role — keep them least-privilege.

66

AWS DevOps (Code*)

medium

What is CodeArtifact?

Answer

CodeArtifact is a managed artifact/package repository for npm, PyPI, Maven, NuGet, etc. It stores your private packages and can proxy/cache public registries through an upstream, giving a single trusted, IAM-controlled source for dependencies and protecting builds from public-registry outages or tampering.

Details & what the interviewer is probing

It's the AWS analogue of Azure Artifacts / Nexus / Artifactory. Proxying public registries adds caching, a security checkpoint, and resilience. Access is IAM/token-based, integrating cleanly with CodeBuild and your developer tooling.

67

AWS DevOps (Code*)

medium

How do the Code* tools integrate with Infrastructure as Code?

Answer

CodePipeline can deploy infrastructure via CloudFormation (or CDK, which synthesizes to CloudFormation) as a deploy action — so app and infra changes flow through the same pipeline. CodeBuild can also run terraform plan/apply. A common pattern: source → build/test → CloudFormation change set (with manual approval) → execute.

Details & what the interviewer is probing

Using CloudFormation change sets with an approval gate gives a reviewable "plan" before infra changes apply — the AWS-native equivalent of reviewing a Terraform plan. This keeps infrastructure changes automated, auditable, and gated rather than hand-applied.

68

AWS DevOps (Code*)

medium

How do you implement blue-green and canary deployments on AWS?

Answer

Use CodeDeploy: for ECS and Lambda it natively shifts traffic canary (a percentage, then the rest after a bake time) or linear (equal increments), and for EC2 it does blue-green by standing up a new fleet behind the load balancer and cutting over. CloudWatch alarms trigger automatic rollback if error/latency metrics breach during the shift.

Details & what the interviewer is probing

The key safety feature is alarm-based auto-rollback during traffic shifting — a bad deploy reverts before most users are affected. For ECS blue-green, an ALB with two target groups swaps traffic; for Lambda, aliases shift weighted traffic. Always pair with health/SLO alarms so the rollout can self-abort.

69

AWS DevOps (Code*)

medium

How do you secure an AWS CI/CD pipeline?

Answer

Give each pipeline/build/deploy action a least-privilege IAM role (not broad admin). For external CI (GitHub Actions/GitLab) deploying to AWS, use OIDC federation instead of stored access keys. Pull secrets from Secrets Manager/SSM at run time (never hardcode), scan images/dependencies in CodeBuild, and gate production with manual approvals.

Details & what the interviewer is probing

The two biggest wins: scoped IAM roles per pipeline action (so a compromised build can't do everything) and OIDC for any external CI (no long-lived keys). Encrypt artifacts (S3 + KMS), enable CloudTrail on pipeline activity, and require approval before prod deploys.

70

VDI

easy

What is VDI, and what are its core components?

Answer

Virtual Desktop Infrastructure (VDI) hosts user desktops/apps as virtual machines on centralized servers in a datacenter/cloud; users connect remotely from any device, and only screen pixels/input travel over the network — the data and compute stay central. Core components: a hypervisor (runs the desktop VMs), a connection broker (authenticates users and brokers them to a desktop), a gateway (secure remote access), a golden/master image (the desktop template), and profile management (user

data/settings).

Details & what the interviewer is probing

The value proposition: data never leaves the datacenter (security/compliance), central management/patching of one image instead of many endpoints, and access from any device (BYOD, contractors). The broker is the brain (load-balances users to hosts); the gateway secures external connections without a VPN.

71

VDI

medium

Persistent vs non-persistent (pooled) desktops — trade-offs?

Answer

Persistent desktops are dedicated — each user keeps their own VM, with changes (apps, files, settings) saved between sessions, like a personal PC. Non-persistent (pooled) desktops are stateless — users get a fresh desktop from the golden image each logon, and it resets on logoff. Non-persistent is far easier to manage and patch (update one image) and denser/cheaper, but requires externalized user profiles/data.

Details & what the interviewer is probing

Most modern deployments favor non-persistent + profile containers (FSLogix) so user identity/data roam onto stateless hosts — best of both worlds: clean managed images + personalized experience. Persistent is used where users need to install their own software or for specialized workstations. The trade-off is management simplicity/cost vs per-user customization.

72

VDI

medium

Describe Azure Virtual Desktop (AVD) architecture.

Answer

AVD is Microsoft's cloud VDI/DaaS. Microsoft manages the control plane (broker, gateway, web access, diagnostics) as a service; you manage the session hosts (the VMs that run desktops/apps), organized into host pools. AVD uniquely supports multi-session Windows (Windows 10/11 Enterprise multi-session) so many users share one VM, cutting cost. Application groups publish full desktops or specific RemoteApps; users are assigned via Entra ID.

Details & what the interviewer is probing

Because Microsoft runs the control plane, you only operate the session-host VMs (size, image, scale). Multi-session Windows is the big cost differentiator vs traditional 1:1 VDI. Pair with FSLogix (profiles on Azure Files), Entra ID + Conditional Access for security, and scaling plans for autoscale. Windows 365 is the simpler, fixed-price "Cloud PC"

(always persistent, per-user) sibling.

73

VDI

medium

What is FSLogix and why is it essential for non-persistent VDI?

Answer

FSLogix stores a user's Windows profile (and Office/OneDrive cache, search index) in a VHD/VHDX container on shared storage (e.g. Azure Files). At logon it mounts the container so the user's full profile appears natively and instantly on whatever stateless session host they land on; at logoff it detaches. This makes non-persistent/pooled desktops feel personalized.

Details & what the interviewer is probing

Without profile containers, roaming profiles on non-persistent hosts are slow and break Office/Outlook/Teams caches. FSLogix solves the core problem of "stateless hosts + personal profile." Watch storage performance/IOPS (profile mounts are I/O sensitive) and size the file share for concurrent logons to avoid logon storms.

74

VDI

medium

What remoting protocols are used in VDI (RDP, HDX, Blast, PCoIP)?

Answer

RDP (Remote Desktop Protocol) — Microsoft's protocol, used by AVD/RDS.

ICA/HDX — Citrix's high-definition protocol, known for performance over poor networks and rich peripheral/multimedia handling.

Blast Extreme / PCoIP — VMware Horizon's protocols (Blast is the modern default; PCoIP legacy).

All transmit display, input, audio, and device redirection efficiently, adapting to bandwidth/latency.

Details & what the interviewer is probing

Protocol quality matters most on high-latency or constrained links — Citrix HDX historically led on WAN performance and graphics; AVD's RDP and VMware Blast have closed much of the gap. For graphics/CAD/GPU workloads, protocol + GPU acceleration (vGPU) is critical. Evaluate by user experience under real network conditions and peripheral/multimedia needs.

75

VDI

medium

VDI vs DaaS, and what are the major platforms?

Answer

VDI traditionally means you build and operate the desktop infrastructure (often on-prem). DaaS (Desktop as a Service) is cloud-delivered VDI where the provider manages the control plane and infrastructure, and you consume desktops as a service. Major platforms: Azure Virtual Desktop and Windows 365 (Microsoft), Amazon WorkSpaces (AWS), Citrix DaaS / Virtual Apps and Desktops, and VMware Horizon.

Details & what the interviewer is probing

DaaS shifts undifferentiated heavy lifting (broker, gateway, scaling infra) to the provider — AVD/Windows 365/WorkSpaces are DaaS; Citrix and VMware can run on-prem (VDI) or as cloud DaaS. Citrix/VMware are often layered on top of AVD/cloud for their advanced management, protocols, and multi-cloud brokering.

76

VDI

medium

What are the main use cases and benefits of VDI?

Answer

Strong fits: security/compliance (data stays in the datacenter, nothing on the endpoint), BYOD and contractors (access from any/unmanaged device safely), remote/distributed workforce, regulated industries (finance, healthcare), GPU workstations (CAD/3D/ML delivered to thin clients), call centers (cheap standardized non-persistent desktops), and M&A / seasonal scaling (rapidly provision/deprovision many desktops).

Details & what the interviewer is probing

Benefits: centralized patching/management of one image, fast onboarding/offboarding, device-independence, and a smaller endpoint attack surface. Trade-offs: dependence on network connectivity, upfront design complexity, and per-user infra cost — VDI isn't cheaper for every workforce, so the use case has to justify it.

77

VDI

medium

How do you size and scale a VDI deployment (e.g. AVD) and manage cost?

Answer

Size by user density per host: profile users as light/medium/heavy (knowledge worker vs CAD), allocate vCPU/RAM/GPU accordingly, and load-test to find users-per-host. Scale with multi-session hosts in host pools plus autoscale/scaling plans that power

hosts on/off by schedule and demand (ramp up before business hours, down at night). Use Spot/reserved capacity and right-sized VM SKUs to control cost.

Details & what the interviewer is probing

The biggest cost lever in cloud VDI is turning hosts off when idle (scaling plans) and packing users via multi-session — running all hosts 24/7 is the classic overspend. Watch for logon storms (everyone logs in at 9am) when sizing storage/compute, and reserve capacity for the steady baseline.

78

VDI

hard

What are common VDI problems and how do you troubleshoot them?

Answer

Slow logons / logon storms — many concurrent logons overwhelm storage (FSLogix profile mounts) or domain controllers. → scale profile storage IOPS, stagger, pre-warm hosts.

Profile bloat/corruption — oversized FSLogix containers, Office cache growth. → size limits, redirect OneDrive, container maintenance.

Poor session performance/latency — network distance, insufficient host resources, no GPU for graphics. → place hosts near users, right-size, add vGPU, tune the protocol.

Capacity/density — too many users per host causing CPU/RAM contention. → re-profile and rebalance.

Details & what the interviewer is probing

Most VDI complaints are storage/IOPS (profiles), network latency, or host over-subscription. Diagnose with the platform's monitoring (AVD Insights / Citrix Director / Horizon) looking at logon duration breakdown, host resource saturation, and protocol round-trip latency. Right-size, place compute near users, and ensure profile storage performance — those three solve the majority of issues.

This guide spans CI/CD, GitOps, configuration management, and VDI for senior engineering interviews. Tool features, defaults, and product availability change across versions (e.g. AWS CodeCommit was restricted to existing customers in 2024; many teams now pair CodePipeline with GitHub/GitLab) — confirm current specifics in vendor docs before an interview. Mastery progress is saved locally in your browser when you open this file directly.